

Exact functional decomposition of polynomials with algebraic coefficients

A unified theory, algorithms, certificates, and implementations

Prepared for Vladimir Reshetnikov
in response to Mathematica StackExchange question 206618

September 2026

Abstract

A polynomial whose coefficients contain radicals or algebraic `Root` objects can be functionally decomposed without factoring its derivative. In characteristic zero, normalizing an inner component to be monic with constant term zero removes the affine ambiguity: its degree alone determines a unique candidate from the leading coefficients of the input. A finite formal-root recurrence constructs it in the coefficient field, and exact polynomial division supplies an acceptance or rejection certificate.

This article synthesizes three technical reports and the accompanying Wolfram Language and Python implementations. We give self-contained proofs of the fixed-degree algorithm, descent through extensions, complete normalized-chain enumeration, and the divisibility poset of all right components. We distinguish two outer-recovery methods and their negative certificates, derive explicit low-degree criteria, and work through radical, nested, quintic `Root`, complex, monomial, and Chebyshev examples. Arithmetic complexity, algebraic representation, the derivative-based answer in the original discussion, the differences among the reports, and current native and independent validation are treated separately.

Contents

1	Problem, scope, and main conclusion	3
2	Normalization and the coefficient field	4
3	The unique right-component candidate	4
3.1	A quadratic-cost formal-root recurrence	5
3.2	The Laurent-series interpretation	5
3.3	Why characteristic zero is explicitly required	6
4	Exact membership in the subalgebra $K[h]$	6
4.1	A second exact recovery method: descending pivots	7
5	Descent, functoriality, and decomposition loci	7
5.1	A coefficient-space interpretation	8
6	One complete chain and all complete chains	8
6.1	Deterministic single-chain algorithm	9
6.2	Enumeration without repeated reassociations	9
7	The finite right-component poset	10
7.1	A field-tower interpretation	11
8	Nonuniqueness and the Ritt perspective	11
8.1	Powers	11

8.2 A Chebyshev collision in the package's normalization	12
9 Exact positive and negative certificates	12
10 Worked examples	13
10.1 A complete quartic criterion	13
10.2 Both possible sextic degree patterns	14
10.3 The original radical polynomial	14
10.4 The nested degree-24 example	15
10.5 An algebraic <code>Root</code> coefficient	15
10.6 Complex coefficients and affine normalization	16
10.7 Composite-degree atoms and exact perturbations	16
11 Comparison and reconciliation of the three reports	16
12 Wolfram Language and Python implementation	17
12.1 Loading and basic operation	17
12.2 Coefficient arithmetic and representation	17
12.3 Public functions	18
12.4 Checking a complete chain with public operations	19
12.5 The Python interface	19
12.6 Algorithmic organization	20
13 Complexity, coefficient growth, and practical limits	20
13.1 Field-operation bounds	20
13.2 Why algebraic arithmetic is not constant-cost	20
13.3 Storage, sparsity, and safe early rejection	21
14 Audit of the derivative-factorization approach	21
15 Validation record and reproducibility	22
15.1 Current native execution	22
15.2 Independent archived checks	23
15.3 The external oracle is a comparison, not a completeness proof	23
15.4 A measured sparse-rejection improvement	24
15.5 Reproduction and trust boundaries	24
16 Conclusions and further implementation work	24

1 Problem, scope, and main conclusion

The motivating question asks whether `Decompose` can be made to work on polynomials with radical coefficients, and presents a derivative-factorization implementation as an alternative [4]. The central distinction is between *functional decomposition*

$$p(x) = f(h(x)), \quad \deg f > 1, \quad \deg h > 1, \quad (1.1)$$

and ordinary multiplicative factorization. They are different problems. Irreducibility under multiplication neither implies nor is implied by indecomposability under composition.

For instance, $x^4 + 1 = (t^2 + 1) \circ x^2$ is functionally decomposable even though it is irreducible over $\mathbb{Q}$. Conversely, $x^4 + x$ is multiplicatively reducible but functionally indecomposable in characteristic zero, as the algorithm below immediately proves. An `Extension` option appropriate for multiplicative factorization is therefore not the fundamental missing ingredient.

Let

$$p(x) = \sum_{i=0}^n a_i x^i, \quad a_n \neq 0, \quad K = \mathbb{Q}(a_0, \dots, a_n) \subseteq \overline{\mathbb{Q}}. \quad (1.2)$$

Because there are finitely many algebraic coefficients, $K/\mathbb{Q}$ is a finite extension. The input expressions choose particular algebraic numbers, not unspecified conjugacy classes. A radical expression, an algebraic `Root`, and an `AlgebraicNumber` can denote the same element of K . Their outward forms should not determine the answer.

The central result. For every proper divisor d of n , there is at most one pair (f, h) satisfying (1.1) with $\deg h = d$, h monic, and $h(0) = 0$. Its only possible h is computed from the leading d coefficients of p by rational operations. A subsequent exact subalgebra-membership test either constructs f or certifies its nonexistence. No polynomial factorization, solving of coefficient equations, extraction of new algebraic numbers, or search through field extensions is needed.

This is a characteristic-zero realization of the classical factorization-free approach to polynomial decomposition, particularly the work of Kozen and Landau [5]. Faster and more general algorithmic treatments exist, including the tame-case work of von zur Gathen [6]. The elementary algorithm here is chosen for transparent proofs, straightforward exact implementation, and verifiable failure certificates, not as a claim to a new asymptotically optimal decomposition algorithm.

What is and is not claimed about the built-in function. The source discussion is from 2019. The currently retrieved Wolfram documentation illustrates symbolic and complex coefficients; it does not state an integer-only mathematical domain for `Decompose` [9]. The historical behavior reported in the question must not be promoted to a verified claim about every current Wolfram version. The native validation reported below concerns the delivered package. It is an independent implementation; it does not modify `System`Decompose` or depend on its behavior.

Scope of the delivered code. The theory applies to any characteristic-zero field with exact arithmetic. The software intentionally accepts the narrower, decidable input class of explicit exact algebraic coefficients recognizable by their respective exact algebraic-number backends. It does not accept approximate coefficients, symbolic parameters, transcendental coefficients, rational functions in x , or finite-characteristic arithmetic. These exclusions are input contracts, not assertions that broader mathematical variants are impossible.

2 Normalization and the coefficient field

Definition 2.1. A nonconstant polynomial $h \in K[x]$ is *normalized* if it is monic and $h(0) = 0$. A decomposition is *nontrivial* if both component degrees exceed one. A *complete chain* is a composition of polynomials of degree at least two, each of which is functionally indecomposable.

Suppose $p = f \circ g$ and write $b = \text{lc}(g) \neq 0$, $c = g(0)$. Set

$$h(x) = \frac{g(x) - c}{b}, \quad F(t) = f(bt + c). \quad (2.1)$$

Then $p = F \circ h$, and h is normalized. This replaces the arbitrary affine freedom at a composition interface by a canonical representative.

More explicitly, if $\ell(t) = bt + c$, the pairs (f, g) and $(f \circ \ell^{-1}, \ell \circ g)$ describe the same composition. Without removing this freedom there are generally infinitely many pairs even for a fixed degree pattern. We enumerate representatives modulo precisely these affine interface changes; we do not enumerate arbitrary choices of linear factors.

In a normalized complete chain

$$p = f_1 \circ f_2 \circ \cdots \circ f_r, \quad (2.2)$$

every component except f_1 is monic with constant term zero. There is no normalization requirement on f_1 : it must carry the input's leading coefficient and constant term. This convention also explains why a correct answer may look different from the radical expression in the question.

Remark 2.2. An algebraic number's selected embedding matters. For example, replacing $\sqrt{2}$ by $-\sqrt{2}$ changes the input polynomial in general. What is representation-independent is replacing an expression by another exact expression for the *same* algebraic number. No **PowerExpand**-style branch assumptions are legitimate here.

3 The unique right-component candidate

Fix $1 < d < n$ with $d \mid n$ and set $m = n/d$. Seek a normalized component

$$h(x) = x^d + b_1 x^{d-1} + \cdots + b_{d-1} x. \quad (3.1)$$

If $p = f(h)$, monicity of h forces $\text{lc}(f) = a_n$. All terms of $f(h) - a_n h^m$ have degree at most $(m-1)d = n-d$. Consequently,

$$[x^{n-k}]h(x)^m = \frac{a_{n-k}}{a_n}, \quad 1 \leq k \leq d-1. \quad (3.2)$$

These are only $d-1$ equations, exactly as many as the unknown coefficients of the normalized h .

Lemma 3.1 (Triangular recovery). *Over a characteristic-zero field, equations (3.2) have a unique solution $b_1, \dots, b_{d-1} \in K$. They can be solved successively by*

$$b_k = \frac{1}{m} \left(\frac{a_{n-k}}{a_n} - [x^{n-k}] \left(x^d + \sum_{j=1}^{k-1} b_j x^{d-j} \right)^m \right). \quad (3.3)$$

Proof. View a choice of $b_j x^{d-j}$ from one of the m factors as a degree deficit of j . A contribution of total deficit k either chooses b_k once and leading terms in every other factor, or uses only deficits smaller than k . The coefficient in the first case is mb_k . No b_j with $j > k$ can contribute. Thus the coefficient equation is linear in the new b_k , with nonzero coefficient m . Induction proves existence and uniqueness. Every step uses arithmetic in K alone. $\square$

Corollary 3.2 (Uniqueness for a fixed degree). *For fixed d , at most one normalized right component can occur. Once it occurs, its outer component is also unique.*

Proof. The first assertion follows from Lemma 3.1. Substitution $K[t] \rightarrow K[x]$, $f \mapsto f(h)$, is injective for nonconstant h , since a nonzero polynomial of degree r maps to one of degree rd . $\square$

3.1 A quadratic-cost formal-root recurrence

Repeatedly expanding powers in (3.3) is conceptually simple but unnecessary. Put

$$S(t) = 1 + \sum_{i=1}^{d-1} s_i t^i \pmod{t^d}, \quad s_i = \frac{a_{n-i}}{a_n}. \quad (3.4)$$

There is a unique $U(t) = 1 + \sum_{k=1}^{d-1} u_k t^k \pmod{t^d}$ with $U(t)^m = S(t) \pmod{t^d}$. It is the finite formal series with constant term one, customarily written $S(t)^{1/m}$. It is not a numerical choice of a complex m th-root branch. Its coefficients are in K .

Differentiating $U^m = S$ in formal power series gives $mSU' = US'$. Comparing coefficients of t^{k-1} yields

$$u_0 = 1, \quad u_k = \frac{1}{mk} \sum_{i=1}^k ((m+1)i - mk) s_i u_{k-i}, \quad 1 \leq k < d. \quad (3.5)$$

For completeness, the left side before isolating u_k is $mk u_k + m \sum_{i=1}^{k-1} (k-i) s_i u_{k-i}$; the right side is $\sum_{i=1}^k i s_i u_{k-i}$. Their difference is precisely (3.5). The candidate is

$$h_d(x) = x^d + u_1 x^{d-1} + \cdots + u_{d-1} x. \quad (3.6)$$

The first three coefficients illustrate the rational nature of the construction:

$$\begin{aligned} u_1 &= \frac{s_1}{m}, \\ u_2 &= \frac{s_2}{m} - \frac{m-1}{2m^2} s_1^2, \\ u_3 &= \frac{s_3}{m} - \frac{m-1}{m^2} s_1 s_2 + \frac{(m-1)(2m-1)}{6m^3} s_1^3. \end{aligned} \quad (3.7)$$

Only terms with indices $k < d$ are needed. The inner constant term is fixed at zero rather than inferred from a formal-root coefficient of order d .

3.2 The Laurent-series interpretation

At infinity, write

$$(p(x)/a_n)^{1/m} = x^d \left(1 + \frac{a_{n-1}}{a_n} x^{-1} + \cdots \right)^{1/m},$$

using the branch whose leading coefficient is one. The positive-degree polynomial part is exactly h_d . If $p = f(h)$ with $f(t) = a_n t^m + c_{m-1} t^{m-1} + \cdots$, then

$$(p/a_n)^{1/m} = h + \frac{c_{m-1}}{ma_n} + O(h^{-1}).$$

Thus the full polynomial part may contain a nonzero constant, but subtracting that constant gives the normalized h . This provides an approximate-root interpretation of the same exact finite computation. It does *not* justify accepting the candidate without testing the lower coefficients of p .

3.3 Why characteristic zero is explicitly required

The triangular construction divides by m ; the efficient recurrence also divides by k . Characteristic zero makes both safe. In characteristic two,

$$x^4 + x^2 = (t^2 + t) \circ x^2 = t^2 \circ (x^2 + x),$$

and the two right components are distinct normalized polynomials of the same degree. Therefore the fixed-degree uniqueness theorem itself can fail in wild characteristic. Even when m is invertible in positive characteristic, the particular recurrence (3.5) cannot simply be used if some $k < d$ is not invertible. The delivered code makes no finite-field claim.

4 Exact membership in the subalgebra $K[h]$

The candidate equations concern only the top of p . To test the whole polynomial, regard h as a polynomial “base.” No unknown outer coefficients or general-purpose solver are required.

Theorem 4.1 (Polynomial-base expansion). *Let $h \in K[x]$ be monic of degree $d \geq 1$. Every $p \in K[x]$ has a unique finite expansion*

$$p(x) = \sum_{j \geq 0} r_j(x) h(x)^j, \quad \deg r_j < d. \quad (4.1)$$

Moreover, $p \in K[h]$ if and only if every r_j is constant.

Proof. Repeated Euclidean division by the monic h gives $q_j = h q_{j+1} + r_j$, starting at $q_0 = p$. Quotient degree drops by d whenever it is at least d , so the process terminates. Substitution gives (4.1).

For uniqueness, suppose a nonzero finite combination $\sum r_j h^j$ vanishes, and let J be the largest index with $r_J \neq 0$. Its leading degree lies in the interval $[Jd, (J+1)d - 1]$, whereas every smaller index contributes a degree below Jd . Cancellation of its leading term is impossible. Hence all digits in a zero combination are zero.

If the digits are constants, the expansion itself defines an outer polynomial in $K[t]$. Conversely, an expression $p = \sum c_j h^j$ with $c_j \in K$ is a valid digit expansion. Uniqueness forces $r_j = c_j$. $\square$

Equivalently, $K[x]$ is a free $K[h]$ -module with basis $1, x, \dots, x^{d-1}$. The nonconstant digit coefficients are the coordinates that prevent membership in its rank-one submodule $K[h] \cdot 1$.

For $\deg p = md$, the digit list is $r_0, \dots, r_m$ and $r_m = a_n$. The outer candidate used by the implementation is

$$F_d(t) = \sum_{j=0}^m r_j(0) t^j. \quad (4.2)$$

It is an actual outer component precisely when all the digits are constant. On rejection the package reports both the nonzero residual $p - F_d(h_d)$ and one nonconstant digit coefficient. The latter identifies the exact failure, not merely an unsuccessful solver run.

Theorem 4.2 (Necessary and sufficient fixed-degree algorithm). *Let K have characteristic zero and let d be a proper divisor of $n = \deg p$. Construct h_d by (3.5)–(3.6), and compute the digits of p in base h_d . There exists a decomposition with right degree d if and only if every digit is constant. On success, (F_d, h_d) is the unique normalized pair.*

Proof. Any normalized right component of degree d must equal h_d by Lemma 3.1. Its existence is therefore equivalent to $p \in K[h_d]$, which Theorem 4.1 decides exactly. The outer component on success is (4.2); its degree is m because $r_m = a_n \neq 0$. Corollary 3.2 supplies uniqueness. $\square$

A small negative example. For $p = x^6 + x$, both proper right degrees are 2 and 3. In either case the leading coefficients force $h_d = x^d$. The digit $r_0 = x$ is nonconstant, so both degrees are impossible. This proves functional indecomposability even though six is composite. The proof works over every characteristic-zero extension of the coefficient field.

4.1 A second exact recovery method: descending pivots

Report [2] instead recovers an outer polynomial by descending coefficient cancellation. Start with $R = p$, and for $j = m, m-1, \dots, 0$ set

$$c_j = [x^{jd}]R, \quad R \leftarrow R - c_j h^j.$$

The final representation is

$$p = \tilde{F}(h) + \tilde{R}, \quad \tilde{F} = \sum_{j=0}^m c_j t^j, \quad [x^{jd}]\tilde{R} = 0 \quad (0 \leq j \leq m). \quad (4.3)$$

It is unique: the leading degree of a nonzero difference of outer polynomials after substitution is divisible by d , whereas a difference of two such residuals has zero coefficients at every divisible degree. Thus $p \in K[h]$ if and only if $\tilde{R} = 0$.

The pivot and digit methods agree on acceptance and on every successful outer component. Their outer candidates and residuals can differ on rejection. For example, take

$$h = x^2 + x, \quad p = h^3 + xh = x^6 + 3x^5 + 3x^4 + 2x^3 + x^2.$$

This h is the forced degree-two candidate. The base- h digits are $(0, x, 0, 1)$, so the digit-constant construction gives $F = t^3$, $R = x^3 + x^2$. Descending pivots instead give $\tilde{F} = t^3 + t$, $\tilde{R} = x^3 - x$. Both are correct negative tests, but they are different certificate schemas. The live interface uses the digit convention of reports [1, 3].

In particular, the nonzero identity $p = G(h) + R$ alone is insufficient as a rejection certificate. A pivot checker must verify the zero-pivot conditions (4.3), while either checker must also verify that h is the uniquely forced normalized candidate.

5 Descent, functoriality, and decomposition loci

Theorem 5.1 (Descent through field extensions). *Let L/K be any extension of characteristic-zero fields and $p \in K[x]$. For any $m, d > 1$, a decomposition of degree pattern (m, d) exists over L if and only if it exists over K . Every normalized pair over L already has both components in $K[x]$.*

Proof. The forward implication is the only one to prove. Normalize a pair over L by (2.1). Its right component is the unique solution of the triangular coefficient system. Formula (3.3) shows inductively that all its coefficients lie in K . Monic division of $p \in K[x]$ by this $h \in K[x]$ then produces digits in $K[x]$. Since they are constant over L , they are elements of K , and these are the coefficients of the outer component. The argument neither assumes that L/K is algebraic nor uses a Galois closure. $\square$

The descent mechanism is a standard consequence of the leading-coefficient method; related formulations appear explicitly in Wyman and Zieve [7]. Its relevance here is particularly strong:

Corollary 5.2. *For the algebraic input field K in (1.2), decomposition over K and over $\overline{\mathbb{Q}}$ gives the same normalized pairs and chains. Enlarging **Extension** cannot create a missing normalized decomposition.*

This conclusion is special to functional decomposition in characteristic zero. It contrasts with ordinary factorization, where $x^2 - 2$ is irreducible over $\mathbb{Q}$ but splits after adjoining $\sqrt{2}$. It also removes any need to decide whether a coefficient's minimal polynomial is solvable by radicals. A quintic `Root` is an entirely ordinary exact coefficient for this algorithm.

Proposition 5.3 (Compatibility with embeddings). *If $\sigma : K \hookrightarrow L$ is a field embedding, then candidate construction, digit expansion, and normalized decomposition commute with σ . In particular, conjugate input polynomials have the same accepted right degrees and the same complete degree patterns.*

Proof. Every construction uses finite additions, multiplications, and divisions by nonzero leading coefficients or nonzero integers. These operations commute with a field embedding. Nonzero coefficients stay nonzero, so the digit obstructions and acceptance decisions agree. $\square$

5.1 A coefficient-space interpretation

The finite tests also describe exactly where decompositions occur in families. Fix $n = md$ and regard $a_0, \dots, a_n$ as coordinates, with a_n invertible. The coefficients of h_d and all its base digits are regular functions in

$$K[a_0, \dots, a_n, a_n^{-1}]$$

when the base field has characteristic zero. In other words, their only variable denominators are powers of the leading coefficient; integer constants in denominators are harmless.

Proposition 5.4 (Fixed-degree decomposition locus). *Within the space of degree- n polynomials, the locus admitting right degree d is the common zero set of the nonconstant digit coefficients. It is isomorphic to the parameter space of arbitrary degree- m outer polynomials and normalized degree- d inner polynomials. In particular it is smooth and irreducible. Its dimension is $m + d$ and its codimension in the $(n + 1)$ -dimensional input coefficient space is $(m - 1)(d - 1)$.*

Proof. Theorem 4.2 gives precisely the stated equations. The composition map from parameters (f, h) is polynomial. It is injective by normalized uniqueness, and its inverse on the locus is the regular candidate-and-digit construction just described. There are $m + 1$ outer coefficients, with the leading one nonzero, and $d - 1$ free inner coefficients. This parameter space is the product of a punctured affine line and an affine space, so it is smooth and irreducible. Hence the dimension is $(m + 1) + (d - 1) = m + d$. Subtracting from $n + 1 = md + 1$ gives $md + 1 - m - d = (m - 1)(d - 1)$. $\square$

This description is not an extra symbolic-parameter feature in the package; it is a mathematical explanation of specialization. As long as degree is preserved, the formulas specialize correctly, but exceptional parameter values may cause all obstructions to vanish. For example, $x^6 + \lambda x$ is indecomposable for nonzero λ and decomposes when $\lambda = 0$. Therefore no numerical “generic behavior” test can replace the exact coefficient tests.

6 One complete chain and all complete chains

Let

$$D(p) = \{d : 1 < d < \deg p, d \mid \deg p, p \in K[h_d]\}.$$

Testing all proper divisors computes every normalized nontrivial pair. There are at most $\tau(n) - 2$ such pairs for degree $n \geq 2$, where $\tau(n)$ is the number of positive divisors of n . Testing only prime right degrees is not complete: $(x^4 + x)^2$ has the atomic degree-four right component $x^4 + x$ and has no right component of degree two. Likewise, there is no justification for restricting the search to $d \leq \sqrt{n}$.

Lemma 6.1 (Minimal right degree gives an atom). *If $D(p)$ is nonempty and d is its smallest member, then h_d is indecomposable.*

Proof. If $h_d = u \circ v$ with both degrees greater than one, then $p = (F_d \circ u) \circ v$. Normalizing v yields a right component of p of degree $1 < \deg v < d$, contradicting minimality. $\square$

6.1 Deterministic single-chain algorithm

Test the proper divisors in increasing order. At the first success, record h_d as the rightmost component and repeat on F_d . Stop when no proper degree succeeds. The outer degree decreases by at least a factor of two at each step. The resulting list is returned outermost first.

Theorem 6.2 (Completeness of the single-chain algorithm). *The algorithm terminates and returns a complete normalized chain for every input of degree at least two. If it returns a singleton, the input is indecomposable over every characteristic-zero extension of K .*

Proof. Termination follows from strictly decreasing positive degree. Each recorded right component is indecomposable by Lemma 6.1. The final outer component is indecomposable because every possible right degree has been rejected by Theorem 4.2. All recorded identities are exact, so substituting them reconstructs p . Descent gives the extension-field claim. $\square$

6.2 Enumeration without repeated reassociations

For all complete chains, it is tempting to split recursively on every pair and on both sides. That repeats the same chain through different binary parenthesizations. The package instead selects the last atom first:

$$\mathcal{C}(p) = \begin{cases} \{(p)\}, & D(p) = \emptyset, \\ \bigcup_{\substack{d \in D(p) \\ h_d \text{ indecomposable}}} \{(c_1, \dots, c_r, h_d) : (c_1, \dots, c_r) \in \mathcal{C}(F_d)\}, & D(p) \neq \emptyset. \end{cases} \quad (6.1)$$

The implementations memoize pair searches and atomicity decisions locally within one call. All-chain traversal is incremental; a requested output limit can stop enumeration without first constructing all remaining chains.

Theorem 6.3 (Complete enumeration modulo affine interfaces). *Formula (6.1) returns exactly all normalized complete chains, with no duplicates caused by binary reassociation. In particular, a specified ordered list of component degrees admits at most one normalized chain.*

Proof. In any normalized complete chain, the final component is indecomposable and has some degree $d \in D(p)$. By fixed-degree uniqueness, it must equal h_d , and the rest of the chain composes to the unique F_d . Induct on degree for the outer chain. This proves both inclusion directions and uniqueness for a fixed ordered degree list. Different selected final degrees cannot produce the same chain, and within one final degree the induction hypothesis rules out duplicates. Strict degree reduction proves finiteness. $\square$

A complete chain has at most $\lfloor \log_2 n \rfloor$ components, but the number of different chains need not be small. Enumerating all of them is necessarily output-sensitive. The interface intentionally separates one chain, all pairs, and all chains so that a user need not request the largest output to solve the ordinary decomposition problem.

Constants and affine polynomials. For software convenience, degree-zero and degree-one inputs return the singleton list containing the input, and the all-chains function returns a list containing that singleton. These are conventions, not complete chains of nonlinear atoms in the definition above. The certificate metadata marks indecomposability as not applicable below degree two. The zero polynomial's reported degree is $-\infty$.

7 The finite right-component poset

Assume throughout this section that $n = \deg p \geq 2$. The degree uniqueness theorem has a stronger consequence that is useful for understanding the whole decomposition structure. Include the two trivial normalized components

$$h_1 = x, \quad h_n = \frac{p - p(0)}{a_n},$$

and let $S(p)$ be their degrees together with all successful proper right degrees.

Theorem 7.1 (Divisibility is the right-component order). *For $d, e \in S(p)$, there exists a polynomial q satisfying $h_e = q \circ h_d$ if and only if $d \mid e$. If it exists, q is unique, monic, has zero constant term, and has degree e/d .*

Proof. The forward implication follows from multiplication of degrees, and uniqueness follows from injectivity of substitution into a nonconstant polynomial. Leading coefficients and constant terms give the normalization of q .

For the converse suppose $d \mid e$ and write $p = g_d(h_d)$, where g_d has leading coefficient a_n and degree n/d . Put $m_e = n/e$. The formal Laurent series

$$V(t) = t^{e/d} \left(\frac{g_d(t)}{a_n t^{n/d}} \right)^{1/m_e}$$

has integer exponents, leading coefficient one, and $V^{m_e} = g_d/a_n$. Its composition with h_d is well defined at infinity and has leading term x^e . Uniqueness of a unit formal root gives

$$V(h_d(x)) = (p(x)/a_n)^{1/m_e}.$$

Every negative power of h_d has only strictly negative powers of x in its Laurent expansion at infinity. Every positive power of h_d is a polynomial with zero constant term. Therefore the strictly positive-power part of $V(h_d)$ is obtained by taking the strictly positive-power polynomial part q of V and substituting h_d . By the Laurent-series formula in Section 3, that positive-power part is h_e . Thus $h_e = q(h_d)$, as required. The argument includes $d = 1$ and $e = n$. $\square$

In particular one can summarize all normalized right components by a finite divisibility poset on at most $\tau(n)$ vertices. Its cover relations encode indecomposable quotients.

Proposition 7.2 (Complete chains as maximal divisor chains). *Let*

$$1 = d_0 \mid d_1 \mid \cdots \mid d_r = n$$

be a strict chain in $S(p)$ and write $h_{d_i} = q_i \circ h_{d_{i-1}}$. Then

$$p = (a_n q_r + p(0)) \circ q_{r-1} \circ \cdots \circ q_1. \quad (7.1)$$

This is a complete decomposition exactly when every adjacent relation is a cover relation in the divisibility poset. All normalized complete decompositions arise this way.

Proof. Composing the quotient identities yields $h_n = q_r \circ \cdots \circ q_1$, which proves (7.1). Suppose some quotient $q_i = A \circ B$ is decomposable. Normalize B ; since q_i is monic with zero constant term, the adjusted A can also be taken monic with zero constant term. Then $B(h_{d_{i-1}})$ is a normalized right component of p whose degree lies strictly between d_{i-1} and d_i and divides d_i . Thus the relation is not a cover.

Conversely, an intermediate degree $e \in S(p)$ with $d_{i-1} \mid e \mid d_i$ gives two quotient identities by Theorem 7.1; their composition factors q_i nontrivially. The outer affine map in (7.1) does not change decomposability. Finally the suffix compositions of any normalized complete chain are normalized right components of p and provide its associated maximal divisor chain. $\square$

This yields an alternative architecture: compute $S(p)$ once, recover quotient labels on its cover edges, and enumerate maximal paths. The live packages use the recursive algorithm of Theorem 6.3; they do not expose a poset or graph API. The poset description is included as a proved structural result and a concrete route to a more efficient future enumerator.

7.1 A field-tower interpretation

For any nonconstant polynomial h of degree d ,

$$[K(x) : K(h(x))] = d. \quad (7.2)$$

Here is an elementary proof. The element x satisfies $h(T) - h(x) = 0$ over $K(h(x))$, so the degree is at most d . If $1, x, \dots, x^{d-1}$ were linearly dependent over $K(h)$, clearing denominators would give a nonzero relation $\sum_{i=0}^{d-1} A_i(h)x^i = 0$ with $A_i \in K[t]$. The nonzero summands have distinct leading degrees modulo d , so their highest terms cannot cancel. This contradiction gives the opposite inequality.

A complete decomposition thus produces a tower

$$K(p(x)) \subset K(f_2 \circ \cdots \circ f_r(x)) \subset \cdots \subset K(f_r(x)) \subset K(x),$$

whose successive degrees are the component degrees. Our algorithms do not construct splitting fields, Galois groups, or these intermediate fields explicitly.

8 Nonuniqueness and the Ritt perspective

Fixed-degree uniqueness does not say that all decompositions have the same ordered degrees. Ritt's characteristic-zero theory describes how complete chains can differ. In particular, their lengths and multisets of component degrees agree, while their order can change. The discussion of Zieve and Müller provides a detailed treatment and modern perspective [8]. None of the proofs of the algorithms above depends on invoking Ritt's classification as a black box.

One consequence of that cited theorem connects completeness with optimization: every complete chain minimizes the largest nonlinear component degree. Indeed, refining a component of degree ab , $a, b > 1$, replaces it by components of smaller degrees a, b , so refinement cannot increase the maximum. Ritt's invariant makes the maximum identical for every complete refinement. This does not imply that all indecomposable degrees are prime: the nested example below contains an indecomposable quartic.

8.1 Powers

For x^{12} , the normalized complete chains are exactly

$$(x^3, x^2, x^2), \quad (x^2, x^3, x^2), \quad (x^2, x^2, x^3).$$

For x^{30} there are six chains, corresponding to permutations of degrees 2, 3, 5. Composing two powers multiplies their exponents:

$$x^r \circ x^s = x^{rs}, \quad \text{not } x^{r+s}. \quad (8.1)$$

Even the correct multiplication should not be used to compress a *complete* chain, because it merges two atoms into a decomposable factor.

In general, if $n = \prod_i p_i^{e_i}$ and $r = \sum_i e_i$, every normalized right component of x^n is x^d for some divisor $d \mid n$. Thus its complete chains are the distinct orders of the multiset of prime factors, numbering $r! / \prod_i e_i!$. For $n = 6, 12, 16, 30, 36, 72$, the counts are 2, 3, 1, 6, 6, 10. These are exact enumeration benchmarks independent of another computer algebra system's decomposition routine.

8.2 A Chebyshev collision in the package's normalization

The identities $T_2 \circ T_3 = T_3 \circ T_2 = T_6$ yield two normalized pairs for

$$T_6(x) = 32x^6 - 48x^4 + 18x^2 - 1 :$$

$$T_6 = (32t^3 - 48t^2 + 18t - 1) \circ x^2,$$

$$T_6 = (32t^2 - 1) \circ \left(x^3 - \frac{3}{4}x\right).$$

Their right degrees are different, so there is no conflict with Corollary 3.2. These are elementary test cases for all-chain enumeration; an implementation that always returns one degree pattern has not enumerated every chain.

More generally, every divisor $d \mid n$ gives the normalized right component

$$H_d(x) = \frac{T_d(x) - T_d(0)}{2^{d-1}}$$

of T_n . The identity $T_a \circ T_b = T_{ab}$ follows by evaluating at $x = \cos \theta$, where both sides equal $\cos(ab\theta)$, and then using equality on infinitely many points. All divisor degrees occur, so the right-component poset and complete-chain count coincide with those of x^n .

More general collisions are not merely commutations of powers. For example, whenever the expressions have the required nontrivial degrees,

$$x^m \circ (x^r R(x^m)) = (x^r R(x)^m) \circ x^m.$$

The package need not recognize such templates in advance: both degree tests succeed directly.

9 Exact positive and negative certificates

A composition identity is a useful positive check, but it does not establish that a failed search was exhaustive or that returned factors are atoms. The package therefore exposes the evidence behind every proposed right degree.

A fixed-degree record contains d, m , the forced h_d , the digit list $r_0, \dots, r_m$, the constant-digit outer candidate F_d , a Boolean outcome, a possible obstruction, and the residual $p - F_d(h_d)$. An obstruction is a triple (j, i, c) with $i > 0$ and

$$c = [x^i]r_j \neq 0.$$

Digit and power indices in the software record are *zero-based* mathematical indices, although Wolfram list positions are one-based. The implementation selects the first nonzero nonconstant coefficient by increasing digit index and then increasing power.

Proposition 9.1 (A certificate checker independent of the search). *A fixed-degree record is sufficient to certify the outcome if a checker verifies all of the following: d is an admissible divisor; h_d is monic of degree d and has constant zero; the leading congruence (3.2) holds; every digit has degree less than d ; the reconstruction (4.1) equals p ; and the recorded status agrees with whether all digits are constant. On rejection, the claimed obstruction must be an actual nonzero nonconstant digit coefficient.*

Proof. The leading congruence and normalization identify the unique possible inner component by Lemma 3.1. Digit reconstruction and degree bounds identify the unique polynomial-base expansion by Theorem 4.1. Its constant or nonconstant nature decides existence. Thus neither the recurrence used to find h_d nor the long-division search must be trusted as part of this checker. $\square$

The delivered verifier follows this strategy: it raises h_d to m by polynomial multiplication and reconstructs the digit expression by Horner composition. It does not call the candidate recurrence, the monic-division routine, or the pair search. It additionally checks that the reported outer candidate is exactly the list of digit constants and that the residual is correct.

An exhaustive record includes a record for *every* proper divisor in increasing order. The verifier checks this entire list, including the absence of omissions and duplicates, and checks the accepted-degree summary. If all tests are negative, it certifies indecomposability. For prime degree the empty divisor list already suffices. To certify a complete returned chain, verify its composition identity and exhaustively certify every nonlinear component as indecomposable. All this can be done through the public API.

Trust boundary. These are mathematical certificates checked by exact computer algebra, not formally verified proof objects in a proof assistant. The search and checker still share scalar algebraic arithmetic and some basic coefficient-vector operations. They therefore do not provide implementation independence against every possible shared bug. The Python implementation uses a different exact number-field representation; the archived checks also compare the recurrence with an independent finite-binomial construction. Current native Wolfram execution and independent checks are reported in Section 15.

10 Worked examples

10.1 A complete quartic criterion

Let

$$p = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0, \quad a_4 \neq 0.$$

There is only one proper degree, $d = 2$. Define

$$b = \frac{a_3}{2a_4}, \quad h = x^2 + bx, \quad c = a_2 - a_4b^2.$$

The recovered outer polynomial and residual are

$$G(t) = a_4t^2 + ct + a_0, \quad R = (a_1 - bc)x.$$

Therefore

$$\boxed{p \text{ decomposes} \iff 8a_4^2a_1 - 4a_4a_3a_2 + a_3^3 = 0.} \quad (10.1)$$

This is valid for arbitrary algebraic coefficients, including nonreal coefficients and nonradical Root values. It follows by direct expansion, so no polynomial factorization or algebraic equation solving is hidden in the criterion.

10.2 Both possible sextic degree patterns

For $p = \sum_{j=0}^6 a_j x^j$, $a_6 \neq 0$, the only possible proper right degrees are 2 and 3. The following outer polynomials and residuals use descending-pivot recovery; on rejection they need not equal the live digit-constant certificate fields.

For $d = 2$, put

$$b = \frac{a_5}{3a_6}, \quad h = x^2 + bx,$$

$$C = a_4 - 3a_6b^2, \quad D = a_2 - Cb^2.$$

Then

$$G(t) = a_6t^3 + Ct^2 + Dt + a_0,$$

$$R(x) = (a_3 - a_6b^3 - 2Cb)x^3 + (a_1 - Db)x.$$

Thus both displayed residual coefficients must vanish.

For $d = 3$, define

$$u = \frac{a_5}{2a_6}, \quad v = \frac{a_4}{2a_6} - \frac{a_5^2}{8a_6^2}, \quad h = x^3 + ux^2 + vx,$$

and $B = a_3 - 2a_6uv$. Then

$$G(t) = a_6t^2 + Bt + a_0,$$

$$R(x) = (a_2 - a_6v^2 - Bu)x^2 + (a_1 - Bv)x.$$

Again the two coefficients give a necessary and sufficient test. Testing only the quadratic inner candidate would miss valid degree pattern (2, 3); testing only the cubic inner candidate would miss pattern (3, 2).

10.3 The original radical polynomial

Put $s = \sqrt{2}$ and take the polynomial in the question:

$$p(x) = 3 + 3s + (14 + 4s)x + (12 + 26s)x^2$$

$$+ (56 + 8s)x^3 + (8 + 48s)x^4 + 48x^5 + 16sx^6. \quad (10.2)$$

For $d = 2$, $m = 3$ and $s_1 = 48/(16s) = 3/s$. Hence $u_1 = 1/s$ and

$$h_2 = x^2 + \frac{x}{s}. \quad (10.3)$$

All base- h_2 digits are constant, giving

$$F_2(t) = 3 + 3s + (8 + 14s)t + (8 + 24s)t^2 + 16st^3. \quad (10.4)$$

Direct substitution verifies $p = F_2(h_2)$. This differs from the question's pair only by the affine normalization $g = sh_2$:

$$g = x + sx^2, \quad f(t) = 3 + 3s + (14 + 4s)t + (4 + 12s)t^2 + 8t^3,$$

and indeed $F_2(t) = f(st)$.

The alternative right degree $d = 3$ produces

$$h_3 = x^3 + \frac{3s}{4}x^2 + \left(\frac{15}{16} + \frac{s}{8}\right)x.$$

Its digit expansion is $p = r_0 + r_1 h_3 + r_2 h_3^2$ with

$$r_0 = 3 + 3s + \left(\frac{51}{16} + \frac{3s}{4}\right)x + \left(\frac{3}{2} + \frac{51s}{16}\right)x^2,$$

$$r_1 = 11 + 2s, \quad r_2 = 16s.$$

The nonconstant r_0 proves that this degree is impossible. Thus the original polynomial has exactly one normalized complete chain, of degrees (3, 2).

```
p = 3 + 3 Sqrt[2] + (14 + 4 Sqrt[2]) x +
    (12 + 26 Sqrt[2]) x^2 + (56 + 8 Sqrt[2]) x^3 +
    (8 + 48 Sqrt[2]) x^4 + 48 x^5 + 16 Sqrt[2] x^6;
chain = AlgebraicDecompose[p, x];
VerifyAlgebraicDecomposition[p, chain, x]
AlgebraicRightDecompose[p, x, 3]
(* Mathematical expectations: True; Missing["NotDecomposable",3]. *)
```

The displayed formulas are exact mathematical results independently checked in the bundled verifier; they are not represented as a native Wolfram session transcript.

10.4 The nested degree-24 example

Let $p_2(x) = p(x^4 - x + 1)$ and set $v = x^4 - x$. Since

$$h_2(v + 1) = v^2 + \left(2 + \frac{1}{s}\right)v + 1 + \frac{1}{s},$$

a normalized chain is

$$H_3(x) = x^4 - x,$$

$$H_2(x) = x^2 + \left(2 + \frac{1}{s}\right)x,$$

$$H_1(x) = 141 + 105s + (168 + 142s)x$$

$$+ (56 + 72s)x^2 + 16sx^3.$$

Thus $p_2 = H_1 \circ H_2 \circ H_3$. The quartic H_3 is indecomposable: its only proposed right degree is two, the leading equations force x^2 , and the term $-x$ obstructs membership. The quadratic and cubic are indecomposable by prime degree. The returned degree sequence is (3, 2, 4). In particular, a component of composite degree can be an atom.

10.5 An algebraic Root coefficient

Let α be the particular real algebraic number selected in Wolfram notation by

```
a = Root[#^5 - # - 1 &, 1];
```

and take

$$q(x) = (x^3 + \alpha x)^2 + (1 + \alpha)(x^3 + \alpha x) + 2.$$

For right degree three the algorithm recovers exactly

$$(t^2 + (1 + \alpha)t + 2, \quad x^3 + \alpha x).$$

No radical formula for α is requested or constructed. The coefficient relation $\alpha^5 - \alpha - 1 = 0$ is available to exact algebraic reduction. The same construction works for a nonreal root of this polynomial and for polynomials with coefficients from several algebraic extensions at once.

A useful validation example adds $(\alpha^5 - \alpha - 1)x^{40}$ to x^4 . The true polynomial is still x^4 , so its degree must be recomputed after coefficient reduction. The package does this before selecting divisors; it does not trust the apparent degree of unreduced input syntax.

10.6 Complex coefficients and affine normalization

Over $\mathbb{Q}(i, \sqrt{2})$,

$$(1 + i)(x^3 + ix)^2 + \sqrt{2} = ((1 + i)t^2 + \sqrt{2}) \circ (x^3 + ix).$$

For a less normalized input, choose

$$f(t) = (1 + \sqrt{2})t^2 + (2 - \sqrt{3})t + 7, \quad g(x) = (2 + \sqrt{3})x^3 + \sqrt{2}x + \sqrt{3}.$$

The unique normalized degree-three pair is

$$\left(f((2 + \sqrt{3})t + \sqrt{3}), \quad x^3 + \frac{\sqrt{2}}{2 + \sqrt{3}}x \right).$$

These examples emphasize that the outer leading coefficient, the original inner leading coefficient, and the original inner constant term are all handled without ad hoc assumptions.

10.7 Composite-degree atoms and exact perturbations

For every $n \geq 2$ and nonzero c , the polynomial $x^n + cx$ is indecomposable over every characteristic-zero extension. Prime degree is immediate. Otherwise every proper right degree satisfies $d \leq n/2$; the leading equations force $h_d = x^d$, and the digit $r_0 = cx$ is nonconstant. The fixed-degree theorem excludes every possible decomposition. Thus $x^n + 10^{-100}x$ is exactly indecomposable although arbitrarily close to x^n . Approximate agreement is not an exact zero certificate.

11 Comparison and reconciliation of the three reports

The three original distributions are preserved byte-for-byte under `polynomial-decompose/reports/`; the live code and this article are separate derived work. All three develop the same normalized-candidate theorem, formal-root recurrence, descent, and finite decomposition algorithms. Their differences matter for implementation and certification.

Report	Distinctive contributions
[1]	<code>AlgebraicDecomposition-1.0.0.zip</code> : monic-division digits, explicit normalization and descent, recursive refinement of both factors, finite enumeration limits, the Ritt minimum-maximum-degree consequence, and the careful order-dependent analysis of derivative recursion. Its independent checks diagnose a missed-decomposition case in the inspected SymPy 1.14.0 implementation.
[2]	<code>algebraic-polynomial-decomposition.zip</code> : descending-pivot outer recovery and residual certificates, explicit quartic and sextic criteria, a self-contained proof of the right-component divisibility poset, and the smooth fixed-degree decomposition locus. Its diagnostic and verification interfaces differ from the digit-based design.
[3]	<code>algebraic_polynomial_decomposition.zip</code> : the digit-certificate API adopted by the live implementation, an independent certificate checker using top congruence and digit reconstruction, deterministic smallest-right-degree recursion, and extensive exact-domain comparisons.

Table 1: The reports agree on the core mathematics but differ in recovery, certificates, enumeration, and software interfaces.

Two mathematical reconciliations are explicit in this article. On rejection, the digit and pivot outer candidates are not interchangeable; their different residuals are illustrated in Section 4. Also, outer-only derivative recursion is not inherently incorrect: report [1] supplies the precise candidate-order condition under which it is justified (Section 14).

All three reports stated that their Wolfram code was unexecuted in the preparation environment. Current native reruns show why that limitation matters. Report 1 passes 55/107 tests, report 2 passes 39/53, and report 3 passes 60/60 under Wolfram 15.0.1. The first two return $\{x^4\}$ as a purported complete decomposition of x^4 , while their pair routines find $\{x^2, x^2\}$; report 2's own completeness verifier also incorrectly accepts the singleton. The diagnosed pair-search control flow uses **Return** inside a **Do** loop as though it exited the enclosing function. The live code adopts report 3's explicit loop-exit structure. These failures concern native implementation, not counterexamples to the mathematical theorem. Independent archived Python checks continue to pass; their narrower evidentiary scope is recorded in Section 15.

12 Wolfram Language and Python implementation

12.1 Loading and basic operation

Load the live Wolfram file from the project directory, with an unassigned polynomial variable:

```
Get["AlgebraicDecomposition.wl"];
Clear[x];
AlgebraicDecompose[(x^4 + x)^2, x]
(* {x^2, x^4 + x}, up to exactly equal formatting *)
```

All factor lists are outermost first. Every component except the first is monic with zero constant term. Neither coefficient formatting nor a choice between radicals and **Root** objects is part of the equality contract. The package does not require Python or an internet connection for evaluation.

12.2 Coefficient arithmetic and representation

The implementation uses ascending dense coefficient vectors. For example, $3 + 2x + x^3$ is represented internally by $\{3, 2, 0, 1\}$. The zero polynomial has the unique vector $\{0\}$. Trailing zero entries are removed only after exact scalar normalization.

The scalar normalization boundary is the private function **red**. Exact integers and rationals pass directly. Other coefficients are processed with **RootReduce**, then checked for exact numeric membership in **Algebraics**. Wolfram documents **RootReduce** as reducing combinations of exact algebraic quantities to a single algebraic-number representation, and **Algebraics** provides the corresponding exact domain membership operation [10, 11]. The algorithm applies these operations to coefficients, not to polynomials containing the free variable.

An approximate real anywhere in the evaluated coefficient expression is rejected. So are nonnumeric symbolic parameters and quantities not certified as algebraic. A failed exact reduction returns a **Failure**; it is never treated as a nonzero coefficient or as evidence of indecomposability. After normalization, equality to the exact integer zero is used for zero tests. There is no tolerance, numerical sampling, or **PossibleZeroQ** in the algorithm.

This contract covers algebraic **Root** objects and exact arithmetic with them, not arbitrary **Root** expressions encoding transcendental equations. It also does not promise to recognize every sophisticated expression that happens mathematically to evaluate to an algebraic number. An explicit algebraic representation can be supplied when the initial expression lies outside the recognized domain.

12.3 Public functions

All lists of factors are outermost first. Mathematical equality, rather than the printed choice between radicals and `Root` objects, is the output contract.

AlgebraicDecompose`[p,x]`. Returns one complete normalized chain. The smallest successful right degree is chosen at each step, giving deterministic degree choices. An input of degree at most one returns its singleton list.

AlgebraicDecompositions`[p,x]`. Returns all normalized complete chains, modulo affine changes at internal interfaces. It implements (6.1), not an enumeration of all raw affine variants or all incomplete chains. The output can be large. The option `"MaxDecompositions" -> Infinity` sets the default unlimited enumeration. A positive integer limit is exceeded only when a further chain is actually found. In that event the result is `Failure["EnumerationLimit",data]`, with `"Complete" -> False`, the requested `"Limit"`, and `"PartialDecompositions"`. Exhausting the search with exactly the limit returns the complete list normally; a truncated list is never silently presented as exhaustive.

AlgebraicDecompositionPairs`[p,x]`. Returns all normalized nontrivial pairs, ordered by increasing right degree. An empty list means no nontrivial pair exists for a valid input. This interface is often a better diagnostic than full chain enumeration.

AlgebraicRightDecompose`[p,x,d]`. Returns the unique normalized pair for a specified proper divisor d . A valid degree with no decomposition returns `Missing["NotDecomposable",d]`. An invalid degree, such as a nondivisor, one, or the whole input degree, returns `Failure`. This distinction separates a proved negative answer from an invalid request.

AlgebraicDecompositionData`[p,x,d]`. Returns the fixed-degree certificate record. Its keys are `"Type"`, `"RightDegree"`, `"OuterDegree"`, `"Inner"`, `"OuterCandidate"`, `"Digits"`, `"Decomposable"`, `"Obstruction"`, and `"Residual"`. The outer candidate is not a valid factor when the status is false. The obstruction is `None` on success, otherwise an association with `"DigitIndex"`, `"Power"`, and `"Coefficient"`.

AlgebraicDecompositionData`[p,x]`. Returns an exhaustive report with `"InputDegree"`, `"TestedRightDegrees"`, `"AcceptedRightDegrees"`, `"Indecomposable"`, and `"Tests"`, in addition to its type marker. For degree below two, the indecomposability field is `Missing["NotApplicable","DegreeBelowTwo"]`.

VerifyAlgebraicDecompositionData`[p,data,x]`. Checks a fixed-degree or exhaustive report using Proposition 9.1. Malformed or inconsistent data returns `False`; invalid polynomial input returns `Failure`. The checker is intended for mathematical data, not as a sandbox for evaluating untrusted Wolfram expressions.

Composition and chain verification. `ComposeDecomposition``[parts,x]` composes a list exactly; the empty list represents the identity x . The default verification call is

```
VerifyAlgebraicDecomposition[p, parts, x]
```

It checks the composition identity. For example, $\{x^4\}$ passes that check for $p = x^4$ but is not a complete nonlinear chain. The options "RequireComplete" $\rightarrow$ True and "RequireNormalized" $\rightarrow$ True add exhaustive atomicity and internal normalization checks, respectively; both default to False. The singleton conventions for inputs of degree at most one are recognized by the completeness option.

12.4 Checking a complete chain with public operations

The following pattern distinguishes the identity check from the atomicity checks:

```
chain = AlgebraicDecompose[p, x];
identityOK = VerifyAlgebraicDecomposition[p, chain, x];
reports = AlgebraicDecompositionData[#, x] & /@ chain;
certificatesOK = And @@ MapThread[
  VerifyAlgebraicDecompositionData[#1, #2, x] &, {chain, reports}];
atomsOK = AllTrue[reports, TrueQ[#"Indecomposable"]] &;
(* For degree(p)>=2, require identityOK && certificatesOK && atomsOK. *)
```

In production code, check for `Failure` before inspecting returned lists or associations. Inputs should use an unassigned symbol as the polynomial variable; ordinary Wolfram evaluation happens before the function patterns are matched.

12.5 The Python interface

The independent live module is `python/algebraic_decompose.py`. Its public functions mirror the mathematical operations:

```
decompose(p, x)
decompositions(p, x, *, max_chains=None)
decomposition_pairs(p, x)
right_decompose(p, x, d)
decomposition_data(p, x, d=None)
verify_decomposition_data(p, data, x)
compose(parts, x)
verify_decomposition(p, parts, x, *,
                    require_complete=False, require_normalized=False)
```

The optional limits and verifier flags are keyword-only parameters. A valid negative `right_decompose` returns `None`; invalid requests raise `ValueError`. Certificate keys use the corresponding snake-case names. The default `max_chains=None` is unlimited. Finding a chain beyond a positive limit raises `EnumerationLimitError`, whose `limit`, `partial_chains`, and `complete=False` attributes make the incomplete result explicit. Exactly attaining a limit is successful when traversal is then exhausted.

An exact SymPy expression or a univariate Poly over QQ or an AlgebraicField is accepted. The implementation chooses one exact coefficient field per call, caches scalar conversions, and performs the polynomial algorithms in that field. Optional FLINT support accelerates rational multiplication and division; it does not change the criterion. A coefficient represented by `CRootOf` is treated as a scalar even when its defining polynomial uses the same printed variable as the input. Approximate coefficients, unbound parameters, and unsupported exact transcendental coefficients are rejected. Both implementations verify the selected exact algebraic numbers, without numeric tolerance.

12.6 Algorithmic organization

The fixed-degree engine consists of three small components: the formal-root recurrence, monic long division, and detection of nonconstant digits. Outer coefficients are taken directly from digit constants. Composition and certificate checking use explicit polynomial convolution and Horner schemes. Loop exits are explicit: the nested obstruction scan uses a locally tagged **Catch/Throw**, and pair search records its result before **Break**. Wolfram’s **Return** inside a **Do** loop exits that loop, not necessarily the surrounding function, so it must not be used as an implicit multilevel exit [14]. There is no call to **Factor**, **Decompose**, **Solve**, or a radical denesting routine in the search. The complete live sources are the loadable **AlgebraicDecomposition.wl** and **python/algebraic_decompose.py**; they are not duplicated as a source appendix, which could diverge from the tested files.

There are deliberately no speculative options for extensions, numerical tolerances, or heuristic fallback solvers. Unsupported input or a failed algebraic normalization does not silently change the mathematical problem. The implementation also does not automatically convert every output to radicals: such a conversion may be impossible or needlessly expensive, and would not improve correctness.

13 Complexity, coefficient growth, and practical limits

13.1 Field-operation bounds

For a fixed proposed right degree d , the recurrence performs $1 + 2 + \dots + (d - 1) = O(d^2)$ scalar operations. A dense division of a polynomial of degree N by a monic degree- d polynomial costs $O((N - d + 1)d)$ field operations. Applying this repeatedly at degrees $n, n - d, n - 2d, \dots$ costs

$$O\left(d \sum_{j=0}^{m-1} (n - jd)\right) = O(n^2), \quad m = n/d.$$

Thus the fixed-degree test is $O(n^2)$ in the dense field-operation model, with $O(n)$ field elements of working storage for its coefficient vectors and digits. This agrees with the classical quadratic fixed-degree approach [5]; it is not a bit-complexity bound.

Testing every proper divisor gives $O(\tau(n)n^2)$ field operations. Along one complete chain, subsequent outer degrees are divisors of n and at most half the previous degree. Since $\tau(n_j) \leq \tau(n)$,

$$\sum_j O(\tau(n_j)n_j^2) \leq O\left(\tau(n)n^2 \sum_{j \geq 0} 4^{-j}\right) = O(\tau(n)n^2).$$

Enumeration of all chains adds the cost of its search graph and its output; no polynomial bound in input size alone is claimed for that interface. Storing all degree certificates simultaneously also costs more than storing one fixed-degree test.

13.2 Why algebraic arithmetic is not constant-cost

Let $D = [K : \mathbb{Q}]$. In a common primitive-element representation $K = \mathbb{Q}(\theta)$, an element is a rational vector of length D , interpreted modulo the minimal polynomial of θ . Multiplication, inversion, and rational height growth all have costs depending on D and coefficient sizes. A concise expression with many independent radicals can generate a field whose degree is the product of their individual degrees. Therefore a polynomial field-operation count does not imply polynomial running time in the length of an arbitrary radical expression.

The delivered Wolfram backend normalizes the input coefficients once and uses a rational fast path, but nonrational arithmetic still calls `RootReduce`. It may repeat expensive algebraic work, especially when many unrelated generators occur. A future Wolfram backend could convert all coefficients to one common field and operate on rational coordinate vectors; `ToNumberField` is relevant to such a design [12]. The live Python implementation already uses one explicit exact coefficient field, with cached scalar conversions and optional FLINT acceleration for rational polynomial arithmetic. Python is not a runtime dependency of the Wolfram package.

13.3 Storage, sparsity, and safe early rejection

Both implementations use dense coefficient vectors. A sparse expression such as $x^{1000000} + x$ still creates a long vector. Within that representation, the live engines avoid unnecessary work: if the leading $d - 1$ normalized coefficients vanish, the candidate is immediately x^d ; monic division skips zero coefficients; and ordinary searches stop at the first nonconstant digit. The certificate interface continues division to retain the full reconstruction. These optimizations preserve Theorem 4.1.

The descending-pivot method likewise costs $O(n^2)$ field operations with ordinary dense arithmetic: successive powers of h cost $O(d^2 m^2) = O(n^2)$, and the eliminations cost $O(nm)$. Storing every power simultaneously requires $O(n^2/d)$ field elements; the digit method uses $O(n)$ working elements for a fixed degree. Retaining all degree certificates and all output chains has additional storage costs. Large exact coefficient heights and the common field degree can dominate either algorithm. A native measurement of one sparse rejection case appears in Section 15; it is not a general speed guarantee.

14 Audit of the derivative-factorization approach

The identity

$$p'(x) = f'(g(x))g'(x)$$

is mathematically useful. In characteristic zero, after suitable scalar normalization, the derivative of a right component divides p' . An exhaustive enumeration of derivative divisors, followed by integration, normalization, and exact membership testing, can therefore produce a correct algorithm. The factorization-free method replaces this potentially large candidate set by one forced candidate per right degree.

Several details in the posted code require correction independently of its choice of method.

One distinct derivative factor is not an obstruction. The early return when the nonconstant factor list has length below two is incorrect. For $p = x^4$, $p' = 4x^3$ has a single distinct nonconstant factor x with multiplicity three, yet $p = x^2 \circ x^2$. Multiplicity cannot be ignored when deciding whether candidate derivatives exist.

Repeated-factor subsets create duplicates. Expanding multiplicities into repeated copies and taking all subsets can visit $2^{\sum e_i}$ subsets, with many yielding the same product. Divisors of $\prod q_i^{e_i}$ are more naturally indexed by multiplicity tuples $0 \leq j_i \leq e_i$, of which there are $\prod (e_i + 1)$. Even that is an avoidable combinatorial search for the present characteristic-zero task.

The candidate degree must divide the input degree. If a proposed derivative has degree $d - 1$, then d must be a proper divisor of n . The posted degree bound on derivative products does not enforce this. Using n/d subsequently as the number of outer coefficients can therefore

produce a noninteger degree and an invalid coefficient system. Divisibility must be checked before constructing the outer polynomial.

Outer-only recursion has an order-dependent justification. A successfully found inner factor is not automatically an atom. However, report [1] correctly observes that an exhaustive derivative search in increasing number of irreducible-factor copies can justify the posted outer-only recursion. If the selected h had $h = A \circ B$, then B' would be a proper divisor of $h' = (A' \circ B)B'$, with fewer factor copies. Its integrated candidate would be a right component of the input and would succeed earlier under complete exact membership testing. Under those conditions, the first successful inner component is indecomposable.

This is a proof obligation on the actual ordering and completeness of the tests. Search caps, a changed order, or an incomplete solver invalidate it. The live single-chain algorithm instead uses increasing right degree and Lemma 6.1, which gives the atom guarantee explicitly.

The final power rewrite changes the composition. The replacement of adjacent powers by x^{n+m} is algebraically wrong; (8.1) requires x^{nm} . For example, adjacent cubic powers compose to x^9 , not x^6 . Correcting addition to multiplication fixes the identity but still destroys completeness by merging nonlinear factors.

Exact checking must remain exact. A heuristic possible-zero predicate is not the same interface contract as coefficientwise exact algebraic reduction. Likewise, the existence of one solver result is not an exhaustive negative certificate when no result is returned. For explicit algebraic coefficients, reduction to canonical exact numbers and the finite obstruction test make the decision boundary explicit.

The historical comment in the source thread describes an old implementation context for `Decompose`; it does not establish that the derivative approach is the only known algorithm. The factorization-free leading-term approach substantially predates that discussion [5, 4].

15 Validation record and reproducibility

15.1 Current native execution

The original native suites were run from temporary copies, preserving all report files. The runtime was Wolfram 15.0.1 on 64-bit Windows.

Package	Passed	Total	Result
Original report 1	55	107	51 wrong-result failures, 1 message failure
Original report 2	39	53	14 wrong-result failures
Original report 3	60	60	Successful
Live package	80	80	Successful

The live suite retains the original report 3 fixtures and adds checks for output limits, certificate corruption, completeness and normalization options, and input contracts. Passing the independent mathematical checks of the first two reports did not prevent their native control-flow failures. A lexical bracket/comment/string scan is likewise not a native execution receipt.

15.2 Independent archived checks

All three archived Python suites were rerun successfully on copies, using Python 3.14.4 and SymPy 1.14.0:

- Report 1 completed 574 checks with no failures, including direct division and finite-binomial candidate comparisons.
- Report 2 completed 144 generated composites, 72 independent triangular candidate comparisons, 348 residual checks, 27 indecomposable-family tests, 28 external rational comparisons, and 16 examples or boundary cases.
- Report 3 covered 974 polynomials, 2,001 fixed-degree tests, 1,042 complete chains, 95 known normalized pairs, and 848 rational comparisons with SymPy. It records 14 differing degree patterns rather than claiming universal agreement.

These categories overlap and are not summed into a count of independent experiments. The archived scripts still print their historical native-unavailable statements; the current native results above are separate evidence. The live cross-language harness passes all 15 common cases: it checks exact substitution, one-chain, all-chain and pair parity, and independently checks Python-generated exhaustive certificates in Wolfram. The cases include the motivating sextic, a nested degree-24 input, the sparse-inner and negative-digit regressions, monomial and Chebyshev collisions, generic and prime-degree inputs, complex and mixed-quadratic coefficients, a quintic root whose bound variable matches the polynomial variable, exact leading-coefficient cancellation, and zero, constant, and linear inputs. The live Python suite passes all 16 test methods, including its generated and boundary cases. All current command lines are recorded in the project README.

A candidate formula independent of the differential recurrence is the finite binomial expansion

$$(1 + w)^{1/m} \equiv \sum_{j=0}^{d-1} \binom{1/m}{j} w^j \pmod{z^d}, \quad w \in zK[z].$$

Constructed composites provide known normalized pairs without asking another decomposition routine to invent the answer. Monomial and Chebyshev chain counts test enumeration independently. Deliberate changes to candidate coefficients, digit lists, accepted-degree summaries, and obstruction fields test that certificate checking rejects malformed evidence.

15.3 The external oracle is a comparison, not a completeness proof

Reports [1, 3] retain missed-decomposition discrepancies from their inspected SymPy 1.14.0 installation. A small example is

$$p = (x^3 + x^2 + x)^2.$$

The external routine with explicit rational-field coefficients returned a singleton, although the construction supplies a cubic inner component. Report 1 diagnoses a missing term in the inspected candidate recurrence; its independent finite-series calculation gives the correct candidate. This is a historical version-specific observation, not a claim about every current SymPy release.

Exact recomposition of a singleton always succeeds. Consequently an external routine's identity check cannot establish that its factors are indecomposable, and agreement with an external singleton is not an indecomposability oracle. The independent degree tests, reconstruction certificates, and known constructions are the completeness checks used here.

15.4 A measured sparse-rejection improvement

In fresh Wolfram 15.0.1 kernels, excluding startup,

```
AlgebraicDecompositionPairs[ $x^{96} + x + 1, x$ ]
```

returns the empty list in both report 3 and the live implementation. A one-second `RepeatedTiming` measurement gave 0.5750906 seconds for report 3 and 0.02458484 seconds for the live code, about a 23.4-fold improvement. This specific sparse negative example benefits from the monomial-candidate shortcut, skipping zero divisor coefficients, and stopping at the first nonconstant digit in ordinary searches. It is not a general speed ratio or a cross-language benchmark.

15.5 Reproduction and trust boundaries

From `polynomial-decompose/`, run the native suite with

```
wolfram -script RunTests.wl
```

The live Python suite and the independent cross-language harness are documented alongside `python/algebraic_decompose.py`. The original report runners remain under their respective `reports/report-0i/` directories. Their output locations belong to the archived layouts, so reproduction should use copies when preserving the report bytes.

Native tests establish that the specified cases execute correctly in the reported runtime. Independent certificates establish the represented mathematical facts using their exact arithmetic backend. Neither is proof-assistant verification of every implementation path, and article PDF rendering is a separate layout check rather than a mathematical test.

16 Conclusions and further implementation work

For explicit exact algebraic coefficients, functional decomposition has a clean finite solution: normalize the inner component, force its coefficients from the top of the input, and check the lower coefficients through polynomial base digits. Each divisor contributes exactly one candidate. Successful pairs descend to the original coefficient field, and rejected candidates carry exact algebraic obstructions. Complete chains and all normalized alternatives follow by elementary recursion.

The live implementations realize this theory without derivative factorization or coefficient solving. Their ordinary searches can stop on the first obstruction, while certificate requests retain complete reconstruction data. Caching, sparse arithmetic, and exact fixed coefficient fields improve practical performance without weakening the decision criterion. The right-component poset gives a proved further route to shared algebraic work and path-based enumeration. These are improvements to engineering and scale, not missing cases in the characteristic-zero mathematical criterion.

References

- [1] *Functional Decomposition of Polynomials with Algebraic Coefficients*. Original report distributed as `AlgebraicDecomposition-1.0.0.zip`; preserved under `polynomial-decompose/reports/report-01/`. Article: `article/article.tex`. Archived source, tests, and oracle notes are retained as provenance, including their historical validation limitations.

- [2] *Exact Functional Decomposition of Polynomials with Algebraic Coefficients*. Original report distributed as `algebraic-polynomial-decomposition.zip`; preserved under `polynomial-decompose/reports/report-02/`. Article: `article/algebraic-decomposition.tex`. The live synthesis uses its triangular recovery, explicit criteria, and right-component-poset arguments.
- [3] *Exact Functional Decomposition over Algebraic Number Fields*. Original report distributed as `algebraic_polynomial_decomposition.zip`; preserved under `polynomial-decompose/reports/report-03/`. Article: `article/article.tex`. The live implementations derive from its digit-certificate interface and correct explicit loop-exit design.
- [4] V. Reshetnikov and contributors, *Is it possible to make Decompose work with coefficients containing radicals?* Mathematica Stack Exchange, question 206618 and answer 206622, 2019. Source question, contributed code, and historical discussion.
<https://mathematica.stackexchange.com/questions/206618/>
- [5] D. Kozen and S. Landau, *Polynomial decomposition algorithms*, Journal of Symbolic Computation **7** (1989), 445–456. [https://doi.org/10.1016/S0747-7171\(89\)80027-6](https://doi.org/10.1016/S0747-7171(89)80027-6)
- [6] J. von zur Gathen, *Functional decomposition of polynomials: the tame case*, Journal of Symbolic Computation **9** (1990), 281–299.
<https://vonzurgathen.online/old/files/gat90c.pdf>
- [7] B. K. Wyman and M. E. Zieve, *Two questions on polynomial decomposition*, Quarterly Journal of Mathematics **63** (2012), 507–511; arXiv:1301.5211. In particular, the coefficient-descent argument in Proposition 2.1. <https://arxiv.org/abs/1301.5211>
- [8] M. E. Zieve and P. Müller, *On Ritt’s polynomial decomposition theorems*, arXiv:0807.3578, 2008. <https://arxiv.org/abs/0807.3578>
- [9] Wolfram Research, *Decompose*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/Decompose.html>
- [10] Wolfram Research, *RootReduce*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/RootReduce.html>
- [11] Wolfram Research, *Algebraics*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/Algebraics.html>
- [12] Wolfram Research, *ToNumberField*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/ToNumberField.html>
- [13] Wolfram Research, *TestReportObject*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/TestReportObject.html>
- [14] Wolfram Research, *Return*, Wolfram Language documentation, especially the example under “Possible Issues.” <https://reference.wolfram.com/language/ref/Return.html>
- [15] SymPy development team, *Polynomial manipulation: reference*, SymPy documentation; local verification used version 1.14.0.
<https://docs.sympy.org/latest/modules/polys/reference.html>